테스트 전략 (Testing Strategy)

개요

Scroll ZKEVM 구현체는 다층적인 테스트 전략을 통해 회로의 정확성과 안정성을 보장합니다. 테스트는 크게 세 가지 레벨로 구성됩니다:

1. **단위 테스트 (Unit Tests)**: 개별 회로와 컴포넌트 검증
2. **통합 테스트 (Integration Tests)**: 실제 블록체인 데이터를 사용한 전체 시스템 검증
3. **공식 테스트 (Official Tests)**: 이더리움 공식 테스트 스위트 실행

단위 테스트 (Unit Tests)

회로별 테스트 구조

각 회로는 자체 테스트 모듈을 포함하며, `#[cfg(any(feature = "test", test))]` 속성으로 조건부 컴파일됩니다.

EVM Circuit 테스트

위치: `zkevm-circuits/src/evm_circuit/test.rs`

EVM Circuit은 캐시된 설정을 사용하여 테스트 성능을 최적화합니다:

```
pub mod cached {
use super::*;
use halo2_proofs::halo2curves::bn256::Fr;
use std::sync::LazyLock;
```

```rust
static CACHE: LazyLock<Cache> = LazyLock::new(|| {
let mut meta = ConstraintSystem::<Fr>::default();
let config = EvmCircuit::<Fr>::configure(&mut meta);
Cache { cs: meta, config }
});

pub struct EvmCircuitCached(EvmCircuit<Fr>);
}
```

주요 테스트 패턴:

- `empty_evm_circuit_no_padding` : 빈 회로 검증
- `empty_evm_circuit_with_padding` : 패딩이 있는 빈 회로 검증
- `get_evm_states_stats` : ExecutionState별 통계 수집
- `get_exec_steps_occupancy` : 셀 사용률 분석

State Circuit 테스트

위치: `zkevm-circuits/src/state_circuit/test.rs`

State Circuit 테스트는 RW 연산의 정렬과 MPT 업데이트 검증에 집중합니다:

```rust
#[cfg(any(feature = "test", test, feature = "test-circuits"))]
mod dev {
// 테스트용 State Circuit 구현
pub use StateCircuit as TestStateCircuit;
```

```
}
```

검증 항목:

- Lexicographic ordering 정확성
- MPT proof type 결정 로직
- State root 전환 정확성
- RW counter 연속성

Mock 데이터 생성

위치: `mock/src/lib.rs`

Mock 크레이트는 테스트용 데이터 생성을 담당합니다:

```
/// Mock 계정 주소
pub static MOCK_ACCOUNTS: LazyLock<Vec<Address>> = LazyLock::new(|| {
vec![
address!("0x0000000000000000000000000000000cafe111"),
address!("0x0000000000000000000000000000000cafe222"),
// ...
]
});

/// Mock 지갑 (서명된 트랜잭션 생성용)
pub static MOCK_WALLETS: LazyLock<Vec<LocalWallet>> = LazyLock::new(|| {
let mut rng = ChaCha20Rng::seed_from_u64(2u64);
```

```
vec![
LocalWallet::new(&mut rng),
LocalWallet::new(&mut rng),
LocalWallet::new(&mut rng),
]
});
```

주요 유틸리티:

- `eth(x)` : ETH 단위 변환
- `gwei(x)` : GWei 단위 변환
- `generate_mock_call_bytecode()` : 컨트랙트 호출 바이트코드 생성
- `MockTransaction` : 트랜잭션 빌더
- `MockAccount` : 계정 상태 빌더
- `MockBlock` : 블록 빌더

TestContext 패턴

위치: `mock/src/test_ctx.rs`

`TestContext` 는 테스트 시나리오를 선언적으로 정의할 수 있게 합니다:

```
let ctx = TestContext::<2, 1>::new(
None, // chain_id
|accs| {
// 계정 설정
accs[0].address(MOCK_ACCOUNTS[0]).balance(eth(20));
```

```
accs[1].address(MOCK_ACCOUNTS[1]).code(bytecode);
},
|mut  txs, accs| {
// 트랜잭션 설정
txs[0]
.from(accs[0].address)
.to(accs[1].address)
.gas_price(gwei(2))
.value(eth(1));
},
|block, _tx| {
// 블록 설정
block.number(0xcafeu64);
},
)
.unwrap();
```

CircuitTestBuilder

위치: `zkevm-circuits/src/test_util.rs`

`CircuitTestBuilder` 는 회로 테스트를 위한 빌더 패턴을 제공합니다:

```
CircuitTestBuilder::new_from_test_ctx(ctx)
.params(CircuitsParams {
```

```
max_evm_rows: 1 << 12,

max_rws: 1 << 12,

..Default::default()

})

.block_modifier(Box::new(|block| {

// 블록 수정

block.circuits_params.max_evm_rows = (1 << 18) - 100;

}))

.evm_checks(Some(Box::new(|prover, gate_rows, lookup_rows| {

// 커스텀 검증 로직

assert_eq!(prover.verify_at_rows_par(

gate_rows.iter().cloned(),

lookup_rows.iter().cloned(),

), Ok(()));

})))

.run();
```

주요 기능:

- `new_from_test_ctx()` : TestContext로부터 생성
- `new_from_block()` : Block으로부터 직접 생성
- `params()` : CircuitsParams 설정
- `block_modifier()` : 블록 수정 함수 추가
- `evm_checks()` : EVM Circuit 검증 로직 커스터마이징

- `state_checks()` : State Circuit 검증 로직 커스터마이징
- `copy_checks()` : Copy Circuit 검증 로직 커스터마이징
- `run()` : 테스트 실행

MockProver 사용

Halo2의 `MockProver` 를 사용하여 회로를 빠르게 검증합니다:

```
let  k = block.get_evm_test_circuit_degree();
let  circuit = EvmCircuit::<Fr>::get_test_cicuit_from_block(block);
let  prover = MockProver::<Fr>::run(k, &circuit, vec![]).unwrap();


// 전체 검증
prover.assert_satisfied_par();


// 특정 행만 검증
let (gate_rows, lookup_rows) = EvmCircuit::get_active_rows(&block);
prover.verify_at_rows_par(
gate_rows.iter().cloned(),
lookup_rows.iter().cloned(),
).unwrap();
```

통합 테스트 (Integration Tests)

구조

위치: `integration-tests/`

통합 테스트는 실제 블록체인 노드와 연동하여 전체 시스템을 검증합니다.

테스트 타입

1. Circuit Tests (`integration-tests/tests/circuits.rs`)

각 회로별로 실제 블록 데이터를 사용한 테스트:

```
macro_rules! declare_tests {
(($name:ident, $block_tag:expr), $real_prover:expr) => {
paste! {
#[tokio::test]
async fn [<serial_test_evm_ $name>]() {
run_test!(EVM_CIRCUIT_TEST, $block_tag, $real_prover);
}

#[tokio::test]
async fn [<serial_test_state_ $name>]() {
run_test!(STATE_CIRCUIT_TEST, $block_tag, $real_prover);
}
// ... 다른 회로들
}
};
}
```

테스트 시나리오:

- `circuit_block_transfer_0` : 기본 전송 트랜잭션
- `circuit_multiple_transfers_0` : 다중 전송
- `circuit_erc20_openzeppelin_transfer_fail` : ERC20 전송 실패 케이스
- `circuit_erc20_openzeppelin_transfer_succeed` : ERC20 전송 성공 케이스
- `circuit_multiple_erc20_openzeppelin_transfers` : 다중 ERC20 전송

2. Mainnet Tests (`integration-tests/tests/mainnet.rs`)

실제 메인넷 블록을 사용한 테스트:

```
# 특정 블록 테스트
GETH0_URL=http://localhost:30303 \
START_BLOCK=11001 \
END_BLOCK=11001 \
cargo test --features=scroll --release test_circuit_all_block

# 특정 트랜잭션 테스트
TX_ID=0xc820f41c097fb21e7d3dcbf450d2e20f28989eea4e36ee2ebd076b6952cf6693 \
GETH0_URL=http://localhost:30303 \
cargo test --features=scroll --release test_mock_prove_tx
```

환경 변수:

- `CIRCUIT` : 테스트할 회로 (`super` , `evm` , `state` , `tx` , `copy` , `keccak` , `none`)
- `GETH0_URL` : RPC 노드 URL
- `START_BLOCK` , `END_BLOCK` : 블록 범위

- `TX_ID` : 특정 트랜잭션 ID

3. RPC Tests (`integration-tests/tests/rpc.rs`)

Geth RPC API 연동 테스트

4. L2 Trace Tests (`integration-tests/tests/l2_trace.rs`)

L2 특화 트레이스 처리 테스트

테스트 데이터 생성

GenDataOutput (`integration-tests/src/lib.rs`):

```rust
#[derive(Serialize, Deserialize)]
pub struct GenDataOutput {
pub coinbase: Address,
pub wallets: Vec<Address>,
pub blocks: HashMap<String, u64>,
pub deployments: HashMap<String, (u64, Address)>,
}
```

블록체인 데이터 생성 후 `gendata_output.json` 에 저장되어 테스트에서 재사용됩니다.

컨트랙트 컴파일

위치: `integration-tests/contracts/`

테스트용 Solidity 컨트랙트:

- `greeter/Greeter.sol` : 기본 컨트랙트
- `ERC20/OpenZeppelinERC20TestToken.sol` : ERC20 토큰

공식 테스트 (Official Tests)

Testool

위치: `testool/`

Testool은 이더리움 공식 테스트 스위트를 실행하는 CLI 도구입니다.

실행 방법

```
# 빌드
cargo build --release

# 기본 테스트 스위트 실행
../target/release/testool --suite default

# Nightly 테스트 스위트 실행
../target/release/testool --suite nightly
```

Oneliner Spec

간단한 테스트를 명령줄에서 직접 실행:

```
# 기본 호출
../target/release/testool --oneliner "call 12;60016002"

# 스토리지 검증
../target/release/testool --oneliner \
```

```
"call;;2000 12;PUSH1(0),SLOAD,CALLVALUE,EQ,PUSH1(1),SSTORE;;00:2000"
```

Spec 형식:

- `txparams: call|create;calldata;value;gas`
- `account: address;code;balance;slot:value`

이더리움 공식 테스트

위치: `testool/tests/src/GeneralStateTestsFiller`

공식 테스트는 다음을 정의합니다:

1. 블록 환경 (blocknumber, timestamp, gaslimit 등)
2. 초기 계정 상태 (balance, nonce, storage, code)
3. 실행할 트랜잭션 세트
4. 실행 후 예상 계정 상태

설정 파일

Config.toml:

```
[[suite]]
id = "default"
max_steps = 10000
max_gas = 0 # 무제한
allow_tests = ["&basic_tests", "specific_test_id"]


[[suite]]
id = "nightly"
```

```
max_steps = 100000

max_gas = 10000000

ignore_tests = ["&problematic_tests"]



[[set]]

id = "basic_tests"

desc = "Basic EVM operations"

tests = ["test1", "test2", "test3"]



[[skip_test]]

tests = ["always_skip_this"]



[[skip_path]]

paths = ["problematic_folder/"]
```

설정 옵션:

- `max_steps` : 최대 opcode 실행 수
- `max_gas` : 최대 가스 (0 = 무제한)
- `allow_tests` : 실행할 테스트 목록 (접두사 `&` 는 테스트 세트)
- `ignore_tests` : 제외할 테스트 목록
- `skip_test` : 항상 건너뛸 테스트
- `skip_path` : 항상 건너뛸 파일/폴더

리포트 생성

```
../target/release/testool --suite default --report
```

생성되는 파일:

- `report/<timestamp>-<git_commit>.html` : 브라우저에서 볼 수 있는 결과
- `report/<timestamp>-<git_commit>.csv` : 원시 결과 데이터

캐시 사용:

```
# 이전 결과 캐시 사용
../target/release/testool --suite default --cache report/previous.csv


# 특정 레벨만 재실행
../target/release/testool --suite default \
--cache report/previous.csv \
--levels fail,panic
```

개별 테스트 디버깅

```
# 특정 테스트 검사
RUST_BACKTRACE=1 ../target/release/testool --inspect <test_id>
```

이 명령은 다음을 제공합니다:

- 테스트 덤프
- Geth 실행 단계
- 상세한 에러 정보

테스트 ID 관리

병렬 실행:

```
# 워커 설정
export WORKER_INDEX=0
export TOTAL_WORKERS=4

# 테스트 ID 파일 읽기
../target/release/testool --test-ids test_ids.txt
```

`read_test_ids()` 함수는 워커 인덱스에 따라 테스트를 분배합니다:

```
pub fn read_test_ids<P: AsRef<Path>>(file_path: P) -> anyhow::Result<Vec<Strin
let worker_index = env::var("WORKER_INDEX")
.ok()
.and_then(|val| val.parse::<usize>().ok())
.expect("WORKER_INDEX not set");
let total_workers = env::var("TOTAL_WORKERS")
.ok()
.and_then(|val| val.parse::<usize>().ok())
.expect("TOTAL_WORKERS not set");
// worker_index에 해당하는 테스트만 필터링
test_ids.enumerate()
.filter_map(|(idx, line)| {
```

```
if idx % total_workers == worker_index {
Some(line)
} else {
None
}
})
.collect()
}
```

테스트 최적화 기법

1. 캐시된 회로 설정

회로 설정(configuration)은 비용이 많이 드는 작업입니다. `LazyLock` 을 사용하여 한 번만 계산:

```
static CACHE: LazyLock<Cache> = LazyLock::new(|| {
let mut meta = ConstraintSystem::<Fr>::default();
let config = EvmCircuit::<Fr>::configure(&mut meta);
Cache { cs: meta, config }
});
```

2. 병렬 검증

`MockProver` 의 병렬 검증 기능 활용:

```
prover.assert_satisfied_par(); // 병렬 검증

prover.verify_at_rows_par(gate_rows, lookup_rows); // 특정 행만 병렬 검증
```

3. Fixed Table 최적화

비트 연산이 필요 없는 테스트는 더 작은 fixed table 사용:

```
fn  detect_fixed_table_tags(block: &Block) -> Vec<FixedTableTag> {

if  need_bitwise_lookup(block) {

FixedTableTag::iter().collect() // 전체 테이블

} else {

FixedTableTag::iter()

.filter(|t| !matches!(t,

FixedTableTag::BitwiseAnd |

FixedTableTag::BitwiseOr |

FixedTableTag::BitwiseXor

))

.collect() // 비트 연산 테이블 제외

}

}
```

4. 동적 행 계산

필요한 최소 행 수만 사용:

```
pub fn get_num_rows_required(block: &Block) -> usize {
let evm_rows = block.circuits_params.max_evm_rows;
if evm_rows == 0 {
Self::get_num_rows_required_no_padding(block)
} else {
block.circuits_params.max_evm_rows + 1
}
}
```

테스트 커버리지

회로별 테스트 범위

1. **EVM Circuit**

- 모든 opcode 실행 상태
- 스택/메모리/스토리지 연산
- 가스 계산
- 에러 케이스

2. **State Circuit**

- RW 연산 정렬
- MPT 업데이트
- State root 전환
- 비존재 증명

3. **Bytecode Circuit**

- 바이트코드 언롤링
- PUSH 명령어 처리
- Code hash 검증

4. **Keccak Circuit**

- Keccak-f[1600] 순열
- 패딩 처리
- 다양한 입력 길이

5. **Tx Circuit**

- 트랜잭션 타입 (PreEip155, Eip155, L1Msg)
- 서명 검증
- RLP 인코딩

6. **Copy Circuit**

- 메모리 복사
- 바이트코드 복사
- 트랜잭션 데이터 복사

7. **고급 회로**

- SHA256 프리컴파일
- MODEXP 프리컴파일
- EC 연산 프리컴파일
- MPT 증명

테스트 메트릭

통계 수집 (`get_evm_states_stats`):

- ExecutionState별 셀 사용량
- 제약 조건 수
- 룩업 횟수
- 행 높이

점유율 분석 (`get_exec_steps_occupancy`):

- Phase 1/2 컬럼 사용률
- 복사 컬럼 사용률
- 룩업 테이블 사용률
- 최적화 기회 식별

디버깅 도구

1. 로깅

```
#[cfg(test)]
#[ctor::ctor]
fn init_env_logger() {
env_logger::Builder::from_env(
env_logger::Env::default().default_filter_or("error")
).init();
}
```

테스트 실행 시:

```
RUST_LOG=debug cargo test
```

2. 회로 시각화

MockProver는 제약 조건 위반 시 상세한 정보 제공:

- 위반된 제약 조건
- 관련 셀 값
- 행 번호

3. 테스트 격리

각 테스트는 독립적으로 실행되어야 합니다:

```
#[tokio::test]
async fn serial_test_evm_transfer() {
// 테스트 로직
}
```

지속적 통합 (CI)

GitHub Actions

테스트는 다음 단계로 실행됩니다:

1. **단위 테스트**

```
cargo test --release --all-features
```

2. **통합 테스트**

```
cargo test --release --features=scroll  -p  integration-tests
```

3. **공식 테스트**

```
cd testool

cargo run --release -- --suite default --report
```

병렬 실행

CI에서 테스트를 병렬로 실행하여 시간 단축:

```
strategy:

matrix:

worker: [0, 1, 2, 3]

env:

WORKER_INDEX: ${{ matrix.worker }}

TOTAL_WORKERS: 4
```

베스트 프랙티스

1. 테스트 작성 가이드라인

- **명확한 테스트 이름**: 테스트가 무엇을 검증하는지 명확히
- **최소 재현 케이스**: 실패를 재현하는 최소한의 코드

- **독립성**: 테스트 간 의존성 없음
- **결정론적**: 항상 같은 결과

2. Mock 데이터 사용

- 실제 데이터와 유사하게
- 엣지 케이스 포함
- 재현 가능한 시드 사용

3. 성능 고려사항

- 캐시된 설정 재사용
- 병렬 검증 활용
- 불필요한 테이블 제외
- 동적 크기 조정

4. 에러 처리

- 명확한 에러 메시지
- 실패 시 컨텍스트 제공
- 디버깅 정보 로깅

관련 문서

- Halo2 통합: Halo2 API 사용법
- 제약 조건 구축: 제약 조건 작성 방법
- Gadget 시스템: 재사용 가능한 컴포넌트

참고 자료

- Halo2 MockProver 문서
- 이더리움 공식 테스트
- Ethereum Tests 문서